

PEACH: recomendación de películas mediante SVD truncada

Trabajo Final — Programación Científica, 2026-1

Universidad Nacional de Colombia, Sede Medellín

Juan Diego Toro Zuluaga • Christian Bustos Betancur • Jack Jarol Jauregui Jauregui • Carlos Daniel Guarín Ortiz

1. Motivación: por qué elegimos este tema

Trabajamos previamente en un sistema de recomendación basado en la descomposición en valores singulares (SVD) en el curso de Álgebra Lineal Aplicada. Allí el énfasis fue algebraico: construir la factorización, truncarla e interpretar los factores. Al terminar, una revisión del trabajo señaló una inconsistencia que no supimos resolver entonces: el informe afirmaba que el modelo optimizaba *sobre las entradas observadas*, mientras que el código ejecutaba la SVD sobre la matriz completa, imputando con cero las calificaciones ausentes. La diferencia no es cosmética: cambia el problema de optimización que realmente se resuelve.

Elegimos retomar el tema porque este curso nos dio exactamente las herramientas que faltaban para tratarlo con rigor computacional y no solo algebraico: condicionamiento y estabilidad de los sistemas lineales que se resuelven, regularización de problemas mal planteados, PCA como proyección a un subespacio de baja dimensión, costo computacional de los métodos iterativos, y diseño de un protocolo experimental con métricas y comparación contra un modelo de referencia. Lo que esperamos mejorar es precisamente eso: enunciar con precisión qué funcional se minimiza en cada etapa, medir en vez de afirmar, y reportar los resultados como son, incluidos los desfavorables.

2. Por qué es interesante en la ciencia

La SVD nace en el siglo XIX en un contexto puramente geométrico. Eugenio Beltrami (1873) y, de forma independiente, Camille Jordan (1874) la obtienen al estudiar las formas bilineales y la reducción de una transformación lineal a ejes canónicos; James J. Sylvester llega a resultados equivalentes en 1889. Erhard Schmidt (1907) da el paso decisivo al extenderla a operadores integrales y demostrar que las sumas parciales de la expansión proporcionan la mejor aproximación de rango finito, resultado que Carl Eckart y Gale Young (1936) formulan en el lenguaje matricial que usamos hoy y que Leon Mirsky (1960) generaliza a toda norma unitariamente invariante. En paralelo, Karl Pearson (1901) y Harold Hotelling (1933) desarrollan el análisis de componentes principales, que es la misma factorización aplicada a datos centrados.

El salto a la ciencia computacional ocurre con Gene Golub y William Kahan (1965), que proponen el primer algoritmo numéricamente estable para calcularla mediante bidiagonalización. Solo entonces la SVD deja de ser un teorema y se convierte en una herramienta: hoy es la base de la pseudoinversa de Moore–Penrose, de los mínimos cuadrados de rango deficiente, de la compresión de datos, del filtrado de ruido y de la reducción de dimensionalidad.

El problema concreto que abordamos —completar una matriz de preferencias mayoritariamente vacía— se volvió central en 2006 con el Netflix Prize, donde los métodos de factorización matricial demostraron ser superiores a los enfoques basados en similitud entre usuarios. Es interesante científicamente porque combina un teorema de aproximación óptima con un problema genuinamente mal planteado: la matriz de calificaciones de MovieLens 32M tiene menos del 0,2% de sus entradas observadas, y aun así hay estructura de bajo rango suficiente para predecir.

3. Ejemplo aplicado: el sistema PEACH

Formulación

Sea $R \in \mathbb{R}^{m \times n}$ la matriz usuario–película y sea $\Omega \subset \{1, \dots, m\} \times \{1, \dots, n\}$ el conjunto de entradas observadas. La SVD de R es $R = U \Sigma V^\top$ con $\sigma_1 \geq \dots \geq \sigma_r > 0$, y su truncamiento de rango k es $R_k = \sum_{i=1}^k \sigma_i u_i v_i^\top$.

Teorema 1 (Eckart–Young–Mirsky). Para todo $k < r$, $\min_{\text{rank}(B) \leq k} \|R - B\|_F = \|R - R_k\|_F = \left(\sum_{i>k} \sigma_i^2 \right)^{1/2}$.

Esto justifica la etapa *offline*: las columnas de $V_k \in \mathbb{R}^{n \times k}$ generan el subespacio k -dimensional que mejor explica los gustos en norma de Frobenius. Aquí está la aclaración que el trabajo anterior omitía: esta minimización recorre *todas* las entradas de R , con los valores ausentes imputados en cero. Es una decisión de modelado, no una optimización sobre Ω , e introduce un sesgo hacia abajo en las películas poco calificadas. La asumimos explícitamente porque permite usar solvers de SVD truncada sobre matrices dispersas (Lanczos implícitamente reiniciado, ARPACK), con costo $\mathcal{O}(\text{nnz}(R)k)$ por iteración, en lugar de un esquema iterativo de completamiento de matrices mucho más costoso.

La etapa *online* sí se restringe a lo observado. Para un usuario nuevo con calificaciones $r \in \mathbb{R}^{|\Omega_u|}$ sobre un conjunto Ω_u de $|\Omega_u| = 5$ películas, sea $S_u \in \mathbb{R}^{5 \times n}$ la matriz de selección de esas filas y $A = S_u V_k \in \mathbb{R}^{5 \times k}$. El perfil latente $x \in \mathbb{R}^k$ resuelve el problema de Tikhonov

$$\min_{x \in \mathbb{R}^k} \|Ax - r\|_2^2 + \lambda \|x\|_2^2 \iff (A^\top A + \lambda I_k) x = A^\top r, \quad (1)$$

y las predicciones son $\hat{r} = V_k x$. La regularización no es opcional: con $k = 50$ y solo 5 observaciones se tiene $\text{rank}(A) \leq 5 < k$, luego $A^\top A$ es singular y el problema sin regularizar tiene infinitas soluciones. Con $\lambda > 0$ la matriz del sistema es simétrica definida positiva y

$$\kappa_2(A^\top A + \lambda I_k) = \frac{\sigma_{\max}(A)^2 + \lambda}{\lambda}, \quad (2)$$

puesto que los $k - \text{rank}(A)$ autovalores restantes valen exactamente λ . La expresión (2) explica lo que observamos numéricamente: el condicionamiento crece como $\mathcal{O}(1/\lambda)$, de modo que λ controla directamente la propagación del ruido de las cinco calificaciones al perfil estimado. El sistema (1) se resuelve por Cholesky en $\mathcal{O}(|\Omega_u|k^2 + k^3)$, despreciable frente a la factorización offline.

Implementación

La implementación se reparte en dos cuadernos con propósitos deliberadamente distintos. `final11_11_12_35.ipynb` contiene el sistema funcional: procesamiento de MovieLens 32M, construcción del modelo por SVD truncada, incorporación de un usuario nuevo a partir de cinco calificaciones, estimación regularizada del perfil latente (1), generación de recomendaciones, perfil de gustos por géneros, búsqueda de usuarios afines y el Mapa de Afinidades. `EvalExpPEACH.ipynb` contiene la validación: además de las métricas reportadas abajo, analiza el espectro singular y la energía capturada, el error relativo de reconstrucción, el condicionamiento de los sistemas y su dependencia de λ según (2), la sensibilidad de las métricas a la dimensión latente k , el costo de la factorización y el tiempo medio de evaluación por usuario. Separar ambos cuadernos fue una decisión de método: impide que la exploración interactiva contamine el protocolo de evaluación.

Protocolo experimental y resultados

Sobre MovieLens 32M separamos 70% de los usuarios para construir V_k y 30% para prueba; de cada usuario de prueba se revelan 5 calificaciones y el resto se mantiene oculto. Con $k = 50$, $\lambda = 0,1$ y 500 usuarios evaluados, comparamos las listas Top-10 de PEACH contra un *baseline* de popularidad.

Métrica (Top-10)	PEACH	Popularidad	Diferencia
HitRate@10	0,7660	0,8240	−0,0580
Precision@10	0,2996	0,3206	−0,0210
Recall@10	0,0785	0,0747	+0,0038
NDCG@10	0,3174	0,3405	−0,0231
Cobertura (títulos distintos)	672	13	×51,7

Tabla 1: Evaluación con 500 usuarios de prueba, $k = 50$, $\lambda = 0,1$.

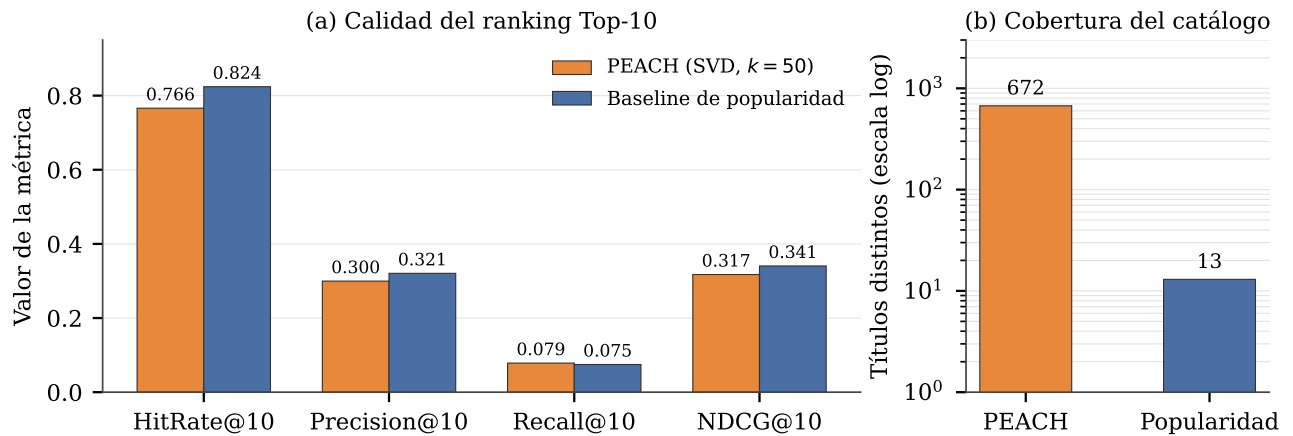


Figura 1: (a) Métricas de calidad del ranking. (b) Cobertura del catálogo en escala logarítmica; sin esa escala la barra del *baseline* sería indistinguible de cero.

El resultado honesto es que el *baseline* gana en tres de las cuatro métricas de ranking. Esto no es un defecto del modelo sino de la métrica: las calificaciones ocultas están fuertemente sesgadas hacia películas populares, así que recomendar los éxitos globales acierta con frecuencia. La diferencia decisiva está en la última fila: el *baseline* recomienda 13 títulos a los 500 usuarios —la misma lista para todos—, mientras PEACH usa 672. Con 5 calificaciones por usuario, PEACH sostiene

una calidad de ranking comparable mientras personaliza de verdad, y las métricas de exactitud por sí solas no capturan esa distinción.

4. Opinión del grupo y autoevaluación

Juan Diego Toro Zuluaga. Para mí lo esencial fue la visualización científica. Antes del curso no sabíamos exponer datos: una gráfica puede ser la misma gráfica y no decir nada, o estar bien construida y tener una base científica sólida. Algo tan sencillo como usar una escala logarítmica cambia por completo la interpretación de un problema —en este mismo informe la figura (b) lo muestra—. Las evaluaciones, prácticas y teóricas, estuvieron bien planteadas y siempre sobre problemas reales.

Christian Bustos Betancur. Lo que más me impactó es que la SVD, que es la herramienta central del proyecto, no sirve solo para esto: aparece en compresión, en filtrado de ruido, en mínimos cuadrados. El curso nos enseñó cosas que resultaron realmente importantes para nuestra vida diaria como programadores y científicos, en particular a usar herramientas de inteligencia artificial de manera adecuada, que hoy es esencial. Es un curso muy completo.

Jack Jarol Jauregui Jauregui. Valoro sobre todo el componente de buenas prácticas y entornos de trabajo. La incorporación de Git y GitHub fue un total acierto: aprendimos a trabajar en equipo en un ambiente colaborativo real. El enfoque orientado a objetos también resultó muy útil, porque nos permite reciclar código y escribirlo de manera mucho más eficiente. Son cosas que quedan y que vamos a seguir usando en materias más avanzadas.

Carlos Daniel Guarín Ortiz. Construir un sistema de recomendación me pareció una idea muy llamativa: no recomienda al azar, sino que a partir de muy poca información predice lo que le puede gustar a cada usuario. Al principio parece magia, y entender que detrás hay una factorización matricial con un teorema de optimalidad lo vuelve más impresionante todavía.

Autoevaluación. Cumplimos el objetivo principal: corregir la inconsistencia del trabajo previo, enunciando con precisión qué se minimiza en cada etapa y validando el modelo con un protocolo explícito. Quedó pendiente el paso natural siguiente: reemplazar la imputación con ceros por un esquema de completamiento que optimice únicamente sobre Ω , lo cual esperamos que mejore las métricas de ranking sin sacrificar la cobertura.